

PLAYWRIGHT

Chapitre 5 — Smart GetBy Locators, Test Runner & Calendars

GetByLabel • GetByRole • GetByText • GetByPlaceholder • filter() • UI Runner • Calendars

À propos de ce chapitre

Ce chapitre introduit une approche plus lisible pour localiser les éléments avec Playwright : `getByLabel()`, `getByRole()`, `getByText()` et `getByPlaceholder()`. Il montre aussi comment combiner ces locators avec `filter()` et comment utiliser le Playwright UI Runner pour observer et déboguer les tests. La dernière partie présente une stratégie d'automatisation et de validation d'un calendrier.

Sommaire

- 1. Objectifs
- 2. GetBy locators
- 3. `getByLabel()`
- 4. `getByPlaceholder()`
- 5. `getByRole()`
- 6. `getByText()`
- 7. `filter()` et chaining
- 8. Playwright UI Runner
- 9. Test E2E avec GetBy
- 10. Limites de `getByLabel()`
- 11. Automatiser un calendrier
- 12. Valider la date
- 13. Exemple complet
- 14. Pièges et bonnes pratiques
- 15. Exercices
- 16. Fiche mémo
- 17. Checklist

1. Objectifs

- Comprendre les locators GetBy.
- Utiliser `getByLabel()` pour les contrôles associés à un label.
- Utiliser `getByPlaceholder()` pour les champs avec placeholder.
- Utiliser `getByRole()` pour les éléments identifiés par leur rôle.
- Utiliser `getByText()` pour cibler un texte.
- Combiner `locator()`, `filter()` et GetBy locators.
- Lancer les tests avec le UI Runner.
- Automatiser année, mois et jour dans un calendrier.
- Valider une date avec `inputValue()` et `expect()`.

2. GetBy locators

Les notes indiquent que la stratégie de locator peut dépendre des pratiques de l'entreprise : `locator(CSS)`, `getByRole`, `getByText`, `getByLabel` ou `getByPlaceholder`.

Locator	Utilisation	Exemple
<code>locator()</code>	CSS/XPath ou locator classique	<code>page.locator(".card-body")</code>
<code>getByLabel()</code>	Label associé à un contrôle	<code>page.getByLabel("Gender")</code>

getByPlaceholder()	Placeholder d'un champ	page.getByPlaceholder("Password")
getByRole()	Rôle + nom accessible	page.getByRole("button",{name:"Submit"})
getByText()	Texte affiché	page.getByText("Success!")

3. getByLabel()

getByLabel() permet de cibler un contrôle de formulaire associé à un label. Le cours l'utilise pour une checkbox, un radio button et un dropdown. [filecite](#)[turn7file0](#)[L17-L30](#)

```
await page.getByLabel("Check me out if you Love IceCreams!").check();
await page.getByLabel("Employed").check();
await page.getByLabel("Gender").selectOption("Female");
```

À retenir : selectOption() s'applique ici au dropdown HTML <select>, comme l'indique le cours.

4. getByPlaceholder()

getByPlaceholder() cible un champ grâce à son attribut placeholder.

```
await page.getByPlaceholder("Password").fill("abc123");

await page.getByPlaceholder("email@example.com")
    .fill("anshika@gmail.com");

await page.getByPlaceholder("enter your passsword")
    .fill("Iamking@000");
```

Cette approche est utilisée dans la réécriture E2E du cours. [filecite](#)[turn7file0](#)[L75-L88](#)

5. getByRole()

getByRole() permet de cibler un élément par son rôle, puis éventuellement par son nom.

```
await page.getByRole("button", { name: "Submit" }).click();
await page.getByRole("link", { name: "Shop" }).click();
await page.getByRole("button", { name: "Login" }).click();
await page.getByRole("button", { name: "Checkout" }).click();
```

Concept : Le paramètre name rend le locator plus précis lorsqu'il existe plusieurs éléments ayant le même rôle.

[filecite](#)[turn7file0](#)[L49-L64](#)

6. getByText()

getByText() permet de retrouver un élément à partir du texte qu'il affiche.

```
await expect(
    page.getByText("Success! The Form has been submitted successfully!.")
).toBeVisible();

await expect(page.getByText("ADIDAS ORIGINAL")).toBeVisible();
await expect(page.getByText("Thankyou for the order.")).toBeVisible();
```

Les notes utilisent getByText() pour les messages de succès, le produit dans le panier et la confirmation de commande. [filecite](#)[turn7file0](#)[L54-L64](#)

7. filter() et chaining

filter() réduit un ensemble d'éléments selon un critère. Le cours l'utilise pour trouver une carte produit contenant un nom, puis rechercher son bouton.

```
await page.locator("app-card")
  .filter({ hasText: "iphone X" })
  .getByRole("button")
  .click();

await page.locator(".card-body")
  .filter({ hasText: "ADIDAS ORIGINAL" })
  .getByRole("button", { name: "Add To Cart" })
  .click();
```

Lecture du second exemple : parmi les .card-body, garder celui contenant ADIDAS ORIGINAL, puis trouver dans cette carte le bouton Add To Cart. [filecite](#)[turn7file0](#)[L92-L101](#)

8. Playwright UI Runner

Le cours présente le UI Runner pour exécuter et observer les tests dans une interface graphique.

```
npx playwright test --ui

node ./node_modules/@playwright/test/cli.js test --ui
```

Le cours montre aussi comment créer un script npm permettant de lancer Playwright avec npm test. [filecite](#)[turn7file0](#)[L7-L16](#)

```
{
  "scripts": {
    "test": "node ./node_modules/@playwright/test/cli.js test"
  }
}

npm test
```

9. Réécriture d'un test E2E avec GetBy

```
import { expect, test } from '@playwright/test';

test("Client App - GetBy Locators", async ({ page }) => {
  await page.goto("https://rahulshettyacademy.com/client");

  // Login using placeholders.
  await page.getByPlaceholder("email@example.com")
    .fill("anshika@gmail.com");

  await page.getByPlaceholder("enter your passsword")
    .fill("Iamking@000");
```

```

// Click Login using its role and name.
await page.getByRole("button", { name: "Login" }).click();

// Wait for products.
await page.waitForLoadState("networkidle");
await page.locator(".card-body b").first().waitFor();

// Find the product card and click Add To Cart.
await page.locator(".card-body")
    .filter({ hasText: "ADIDAS ORIGINAL" })
    .getByRole("button", { name: "Add To Cart" })
    .click();

// Open the cart.
await page.getByRole("listitem")
    .getByRole("button", { name: "Cart" })
    .click();

// Verify product.
await expect(page.getByText("ADIDAS ORIGINAL")).toBeVisible();

// Checkout.
await page.getByRole("button", { name: "Checkout" }).click();

// Type the first characters of the country.
await page.getByPlaceholder("Select Country")
    .pressSequentially("Fra", { delay: 100 });

// Wait for autocomplete results.
await page.locator(".ta-results").waitFor();

// Select the first matching option.
await page.getByRole("button", { name: "Fra" }).nth(0).click();

// Place the order.
await page.getByText("PLACE ORDER").click();

// Validate confirmation.
await expect(
    page.getByText("Thankyou for the order.")
).toBeVisible();
});

```

Ce scénario reprend les étapes du cours : login, attente des produits, filtre sur ADIDAS ORIGINAL, panier, checkout, recherche du pays, puis validation de la commande. [filecite](#) [turn7file0](#) [L78-L117](#)

10. Limites de getByLabel()

Le cours précise que getByLabel() fonctionne lorsque le champ est correctement associé au label. Deux structures sont données.

```

<label>
  Password
  <input type="password" />

```

```

</label>

<label for="passwordField">
    Password
</label>
<input id="passwordField" type="password" />

```

Idée : Dans les deux cas, le label est relié au contrôle. Si cette relation n'existe pas dans le HTML, getByLabel() ne doit pas être supposé fonctionner.

11. Automatiser un calendrier

Le cours utilise le calendrier de <https://rahulshettyacademy.com/seleniumPractise/#/offers> et définit une date attendue.

```

const day = "7";
const month = "7";
const year = "2026";

await page.goto(
    "https://rahulshettyacademy.com/seleniumPractise/#/offers"
);

```

11.1 Ouvrir le calendrier

```
await page.locator(".react-date-picker__inputGroup").click();
```

11.2 Aller à la sélection de l'année

```

await page.locator(".react-calendar__navigation__label").click();
await page.locator(".react-calendar__navigation__label").click();

```

11.3 Sélectionner l'année

```
await page.getByText(year).click();
```

11.4 Sélectionner le mois

Le cours rappelle que month = 7 correspond à July et utilise nth(month - 1), car nth() est indexé à partir de 0.

```

await page.locator(
    ".react-calendar__year-view__months__month"
).nth(Number(month) - 1).click();

```

Mois	Index
1	0
2	1
7 (juillet)	6
12	11

11.5 Sélectionner le jour

```
await page.locator("//abbr[text()='\" + day + "\']").click();
```

Les notes indiquent qu'un locator CSS basé sur les classes du bouton du calendrier n'a pas fonctionné dans cet exemple, puis utilisent cet XPath. ☐ filecite ☐ turn7file0 ☐ L135-L143 ☐

12. Valider la date

Après la sélection, le cours lit les trois inputs du date picker et compare leurs valeurs aux valeurs attendues.

```
const expectedList = [month, day, year];

const inputs = page.locator(
  ".react-date-picker__inputGroup__input"
);

for (let i = 0; i < expectedList.length; i++) {
  const value = await inputs.nth(i).inputValue();
  expect(value).toEqual(expectedList[i]);
}
```

Cette logique est explicitement présentée dans la partie de validation du calendrier. [filecite turn7file0 L145-L176](#)

Point important : `inputValue()` lit la valeur d'un champ input ; `expect(...).toEqual(...)` compare cette valeur avec la valeur attendue.

13. Exemple complet : calendrier

```
import { expect, test } from '@playwright/test';

test("Calendar validations", async ({ page }) => {
  const day = "7";
  const month = "7";
  const year = "2026";

  await page.goto(
    "https://rahulshettyacademy.com/seleniumPractise/#/offers"
  );

  // Open date picker.
  await page.locator(
    ".react-date-picker__inputGroup"
  ).click();

  // Open year selection.
  await page.locator(
    ".react-calendar__navigation__label"
  ).click();

  await page.locator(
    ".react-calendar__navigation__label"
  ).click();

  // Select year.
  await page.getByText(year).click();

  // month "7" -> nth(6), because nth() starts at 0.
  await page.locator(
    ".react-calendar__year-view__months__month"
  ).nth(Number(month) - 1).click();
```

```

// Select day.
await page.locator(
  "//abbr[text()='\" + day + '\"]"
).click();

// Validate all three date fields.
const expectedList = [month, day, year];
const inputs = page.locator(
  ".react-date-picker__inputGroup__input"
);

for (let i = 0; i < expectedList.length; i++) {
  const value = await inputs.nth(i).inputValue();
  expect(value).toEqual(expectedList[i]);
}
});

```

14. Pièges et bonnes pratiques

getByLabel() : Vérifier que le label est réellement associé au contrôle.

getByPlaceholder() : Le placeholder doit correspondre exactement au champ visé.

getByRole() : Utiliser name pour désambiguïser les éléments ayant le même rôle.

getByText() : Utiliser un texte suffisamment stable pour éviter les locators fragiles.

filter() : Filtrer le composant parent avant de chercher son bouton ou autre élément enfant.

nth() : Le premier élément est nth(0), pas nth(1).

Calendrier : La stratégie XPath du cours dépend de la structure HTML de ce calendrier.

Attentes : Dans le scénario client, le cours utilise waitForLoadState('networkidle') puis attend le premier titre produit.

UI Runner : Utiliser --ui pour observer et déboguer pendant l'apprentissage.

15. Exercices d'entraînement

1. Réécrire un locator CSS de champ en getByLabel().
2. Réécrire un champ avec getByPlaceholder().
3. Réécrire un bouton avec getByRole().
4. Réécrire une vérification de message avec getByText().
5. Créer un locator avec locator().filter({ hasText: ... }).getByRole(...).
6. Changer le produit ADIDAS ORIGINAL par un autre produit dans le test E2E.
7. Modifier le pays recherché dans le checkout.
8. Choisir une autre année dans le calendrier.

9. Choisir un autre mois et vérifier le calcul de l'index.
10. Ajouter une assertion supplémentaire après la sélection du jour.
11. Lancer le test avec `npx playwright test --ui` et observer les étapes.

16. Fiche mémo

Besoin	Syntaxe
Label	<code>page.getByLabel("Gender")</code>
Placeholder	<code>page.getByPlaceholder("Password")</code>
Role + nom	<code>page.getByRole("button", { name: "Submit" })</code>
Texte	<code>page.getByText("Success!")</code>
Filtrer	<code>locator(".card").filter({ hasText: "Product" })</code>
Premier	<code>locator.first()</code>
Index	<code>locator.nth(i)</code>
Select	<code>locator.selectOption("Female")</code>
Lire input	<code>await locator.inputValue()</code>
Visible	<code>await expect(locator).toBeVisible()</code>
UI Runner	<code>npx playwright test --ui</code>
Pause	<code>await page.pause()</code>
Attendre réseau	<code>await page.waitForLoadState("networkidle")</code>
Attendre locator	<code>await locator.waitFor()</code>

17. Checklist de validation

- ☐ Je comprends le rôle d'un locator.
- ☐ Je sais utiliser `getByLabel()`.
- ☐ Je sais reconnaître les deux associations label/contrôle montrées dans le cours.
- ☐ Je sais utiliser `getByPlaceholder()`.
- ☐ Je sais utiliser `getByRole()` avec `name`.
- ☐ Je sais utiliser `getByText()`.
- ☐ Je comprends le chaining de locators.
- ☐ Je sais utiliser `filter({ hasText: ... })`.
- ☐ Je sais lancer le UI Runner.
- ☐ Je sais sélectionner une année, un mois et un jour dans le calendrier du cours.
- ☐ Je comprends pourquoi le mois 7 correspond à `nth(6)`.
- ☐ Je sais lire un input avec `inputValue()`.
- ☐ Je sais valider une valeur avec `expect().toEqual()`.

Conclusion

Le chapitre 5 te fait passer d'une recherche principalement basée sur les sélecteurs CSS à une approche plus descriptive avec les GetBy locators. Les concepts à maîtriser sont `getByLabel()`, `getByRole()`, `getByText()`, `getByPlaceholder()`, `filter()` et le chaining. La partie calendrier ajoute une compétence importante : décomposer un composant complexe en étapes puis valider le résultat avec `inputValue()` et des assertions.

Les 5 notions essentielles

- `getByRole()` + `name` pour cibler précisément les boutons et liens.
- `getByLabel()` pour les contrôles associés à un label.
- `getByPlaceholder()` pour les champs identifiés par leur placeholder.
- `filter({ hasText: ... })` pour cibler un composant dans une collection.
- `nth()` est zéro-based et `inputValue()` permet de lire la valeur d'un input.